Степени матрицы

Пусть A — квадратная матрица $n \times n$. Тогда A можно умножить на себя и получить матрицу A^2 того же размера. Затем, получившуюся матрицу снова можно умножить на A и т.д. Таким образом можно определить степень A^k, $k \in \mathbb{N}$. При этом порядок умножений не важен за счёт ассоциативности умножения.

Положим $A^0 = E$. А также $A^{-k} = (A^{-1})^k$. Легко убедиться, что степени матрицы удовлетворяют стандартным свойствам степеней чисел: $A^k A^m = A^{k+m}$ и $(A^k)^m = A^{km}$.

В частности, верно, что $(A^{-1})^k = (A^k)^{-1}$.

Многочлен от матрицы

Пусть $f(x) = c_m x^m + c_{m-1}x^{m-1} + \ldots + c_0$ — многочлен. Можно записать его так: $f(x) = c_m x^m + c_{m-1}x^{m-1} + \ldots + c_0 x^0$. Тогда можно применить многочлен к квадратной матрице A:

$$f(A) = c_m A^m + c_{m-1}A^{m-1} + \ldots + c_0 A^0 =$$

$$= c_m A^m + c_{m-1}A^{m-1} + \ldots + c_0 E.$$

Заметим, что при умножении $c_k A^k$ также получается матрица $n \times n$ и такие матрицы можно сложить.

Так введённые многочлены от матрицы удовлетворяют, в частности, соотношению $f(A)g(A) = (fg)(A)$, где fg — обычное произведение многочленов.

След матрицы

Пусть A — квадратная матрица. Следом этой матрицы называется число $\mathrm{tr}(A)$, равное сумме элементов на главной диагонали. То есть $\mathrm{tr}A = \sum\limits_{i=1}^{n} a_{ii}$.

Теорема 1. *1)*

$$\mathrm{tr}(A + B) = \mathrm{tr}(A) + \mathrm{tr}(B)$$

2)

$$\mathrm{tr}(A^T) = \mathrm{tr}(A)$$

3)

$$\mathrm{tr}(AB) = \mathrm{tr}(BA)$$

4)

$$\mathrm{tr}(CAC^{-1}) = \mathrm{tr}(A)$$

Доказательство

1) $\mathrm{tr}(A+B) = \mathrm{tr}(A) + \mathrm{tr}(B)$

Пусть $A + B = C$. Тогда $c_{ij} = a_{ij} + b_{ij}$. Имеем

$$\mathrm{tr}(A+B) = \mathrm{tr}C = \sum_{i=1}^{n} c_{ii} = \sum_{i=1}^{n}(a_{ii} + b_{ii}) = \sum_{i=1}^{n} a_{ii} + \sum_{i=1}^{n} b_{ii} = \mathrm{tr}A + \mathrm{tr}B.$$

2) $\mathrm{tr}(A^T) = \mathrm{tr}(A)$

Действительно, на диагонали у A^T стоят просто те же числа, что и у A. Поэтому и их сумма та же самая.

Доказательство

3) $\mathrm{tr}(AB) = \mathrm{tr}(BA)$

Пусть $AB = C$, $BA = D$. Тогда $c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}$, $d_{ij} = \sum_{k=1}^{n} b_{ik}a_{kj}$. Получаем

$$\mathrm{tr}\, C = \sum_{i=1}^{n} c_{ii} = \sum_{i=1}^{n}\sum_{k=1}^{n} a_{ik}b_{ki} = \sum_{k=1}^{n}\sum_{i=1}^{n} a_{ik}b_{ki} = \sum_{k=1}^{n}\sum_{i=1}^{n} b_{ki}a_{ik} = \sum_{k=1}^{n} d_{kk} = \mathrm{tr}D.$$

4) $\mathrm{tr}(CAC^{-1}) = \mathrm{tr}(A)$

$$\mathrm{tr}(CAC^{-1}) = \mathrm{tr}((CA)(C^{-1})) = \mathrm{tr}((C^{-1})(CA)) = \mathrm{tr}\, A.$$

Определение 2. Пусть $A \in \mathrm{Mat}_{m,n}$. Матрица $B \in \mathrm{Mat}_{n,m}$ называется <u>левой обратной к матрице A</u>, если $BA = E(= E_n)$. Если такая матрица существует, будем говорить, что матрица A <u>обратима слева</u>.

Матрица $C \in \mathrm{Mat}_{n,m}$ называется <u>правой обратной к матрице A</u>, если $AC = E(= E_m)$. Если такая матрица существует, будем говорить, что матрица A <u>обратима справа</u>.

Определение

Если у матрицы A есть левая обратная, совпадающая с правой обратной, она называется <u>обратной матрицей к матрице A</u> и обозначается A^{-1}. Матрица A при этом называется <u>обратимой</u>.

Замечание

Если $m \neq n$, то левая/правая обратная (если существует) будет определена не однозначно. Например, левыми обратными к матрице

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}$$

будут как

$$B = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

так и

$$B' = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Обратная матрица

Если $m \neq n$, то хотя бы с одной стороны (слева или справа) матрица A не обратима (будет доказано позже, когда будем изучать ранги). Рассмотрим случай квадратной матрицы $A \in \mathrm{Mat}_{n,n}$.

Теорема 3. *Пусть у матрицы A есть левая обратная B и правая обратная C. Тогда $B = C$.*

Доказательство. $C = EC = (BA)C = B(AC) = BE = B$. □

Следствие 4. *Если обратная матрица существует, то она единственная.*

Доказательство. Пусть B и C — две различные обратные матрицы к матрице A. Тогда B в частности левая обратная, а C — правая обратная. Значит, $B = C$. □

Треугольные матрицы

Если у матрицы все элементы ниже главной диагонали равны нулю ($\forall i > j \colon a_{ij} = 0$), то матрицу называют *верхнетреугольной*.

$$\begin{pmatrix} 3 & 7 & 8 & -5 \\ 0 & 2 & 1 & -1 \\ 0 & 0 & 0 & 3 \\ 0 & 0 & 0 & -2 \end{pmatrix}$$

Аналогично, если все элементы выше главной диагонали равны нулю, то матрицу называют *нижнетреугольной*.

Блочные матрицы

Иногда матрица может быть задана несколькими своими «кусками» по отдельности, например: $M = \begin{pmatrix} A & B & C \\ D & F & H \end{pmatrix}$, где A, B, C, D, F, H заданы отдельно (для этого их размеры должны, конечно, правильно согласовываться). Говорят, что M — блочная матрица. Такие матрицы естественно возникают, если мы искусственно объединили какие-то наборы данных вместе, чтобы дальше их вместе обрабатывать. Или, наоборот, одну большую матрицу в некоторых алгоритмах бывает удобно разбить на блоки меньшего размера, с каждым из которых действия будут происходить по отдельности (см. далее).

Блочные матрицы: сложение

Из определения сложения (и умножения на число) очевидно следует, что блочные матрицы одинаковой структуры складываются покомпонентно, например:

$$\begin{pmatrix} A \\ B \end{pmatrix} + \begin{pmatrix} C \\ D \end{pmatrix} = \begin{pmatrix} A + C \\ B + D \end{pmatrix},$$

где размеры A и C должны совпадать, размеры B и D должны совпадать, а у всех подматриц должна совпадать ширина.

Блочное умножение

Более удивительно, что умножение матриц тоже может выполняться блочно.

Например, выполнено равенство

$$\begin{pmatrix} A & B \\ C & D \end{pmatrix} \begin{pmatrix} F & G \\ H & K \end{pmatrix} = \begin{pmatrix} AF + BH & AG + BK \\ CF + DH & CG + DK \end{pmatrix}.$$

Блочное умножение

Теорема 5. *Пусть* $M = \begin{pmatrix} M_{11} & M_{12} & \dots & M_{1s} \\ M_{21} & M_{22} & \dots & M_{2s} \\ \vdots & \vdots & \ddots & \vdots \\ M_{r1} & M_{r2} & \dots & M_{rs} \end{pmatrix}$, $N = \begin{pmatrix} N_{11} & N_{12} & \dots & N_{1t} \\ N_{21} & N_{22} & \dots & N_{2t} \\ \vdots & \vdots & \ddots & \vdots \\ N_{s1} & N_{s2} & \dots & N_{st} \end{pmatrix}$, *где матрица* M_{ij} *имеет размер* $m_i \times n_j$, *а матрица* N_{jl} *имеет размер* $n_j \times k_l$, *причём* $\sum m_i = m, \sum n_j = n, \sum k_l = k$. *Тогда для произведения* MN *выполнено равенство*

$$MN = \begin{pmatrix} \sum\limits_{j=1}^{s} M_{1j}N_{j1} & \sum\limits_{j=1}^{s} M_{1j}N_{j2} & \dots & \sum\limits_{j=1}^{s} M_{1j}N_{jt} \\ \sum\limits_{j=1}^{s} M_{2j}N_{j1} & \sum\limits_{j=1}^{s} M_{2j}N_{j2} & \dots & \sum\limits_{j=1}^{s} M_{2j}N_{jt} \\ \vdots & \vdots & \ddots & \vdots \\ \sum\limits_{j=1}^{s} M_{rj}N_{j1} & \sum\limits_{j=1}^{s} M_{rj}N_{j2} & \dots & \sum\limits_{j=1}^{s} M_{rj}N_{jt} \end{pmatrix}.$$

Блочное умножение

Пример 6.

$$\left(\begin{array}{cc|c} 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ 7 & 8 & 9 \end{array}\right) \left(\begin{array}{c|cc} 10 & 20 & 30 \\ \hline 40 & 50 & 60 \\ \hline 70 & 80 & 90 \end{array}\right) =$$

$$= \left(\begin{array}{c|cc} 1 \cdot 10 + 2 \cdot 40 + 3 \cdot 70 & 1 \cdot 20 + 2 \cdot 50 + 3 \cdot 80 & 1 \cdot 30 + 2 \cdot 60 + 3 \cdot 90 \\ \hline 4 \cdot 10 + 5 \cdot 40 + 6 \cdot 70 & 4 \cdot 20 + 5 \cdot 50 + 6 \cdot 80 & 4 \cdot 30 + 5 \cdot 60 + 6 \cdot 90 \\ 7 \cdot 10 + 8 \cdot 40 + 9 \cdot 70 & 7 \cdot 20 + 8 \cdot 50 + 9 \cdot 80 & 7 \cdot 30 + 8 \cdot 60 + 9 \cdot 90 \end{array}\right) =$$

$$= \begin{pmatrix} \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 10 \\ 40 \end{pmatrix} + \begin{pmatrix} 3 \end{pmatrix} \begin{pmatrix} 70 \end{pmatrix} & \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 20 & 30 \\ 50 & 60 \end{pmatrix} + \begin{pmatrix} 3 \end{pmatrix} \begin{pmatrix} 80 & 90 \end{pmatrix} \\ \begin{pmatrix} 4 & 5 \\ 7 & 8 \end{pmatrix} \begin{pmatrix} 10 \\ 40 \end{pmatrix} + \begin{pmatrix} 6 \\ 9 \end{pmatrix} \begin{pmatrix} 70 \end{pmatrix} & \begin{pmatrix} 4 & 5 \\ 7 & 8 \end{pmatrix} \begin{pmatrix} 20 & 30 \\ 50 & 60 \end{pmatrix} + \begin{pmatrix} 6 \\ 9 \end{pmatrix} \begin{pmatrix} 80 & 90 \end{pmatrix} \end{pmatrix}$$

По размерам получается $((1+2) \times (2+1)) \cdot ((2+1) \times (1+2)) = ((1+2) \times (1+2))$.

Блочное умножение

Столкнувшись с таким утверждением, естественно догадаться, что оно должно доказываться напрямую, простой проверкой, что именно получается при перемножении матриц. Но такая проверка может оказаться очень утомительна, так, при попытке посчитать (i, j)-тую клеточку произведения блочных матриц мы столкнёмся с необходимостью как-то обозначить, в какие именно блоки попадает i-тая строка матрицы M и j-тый столбец матрицы N... Все эти проблемы преодолимы, но мы воспользуемся чуть другой *стратегией*: разобьём задачу на части и докажем постепенно.

Лемма 1.

$$\begin{pmatrix} A_1 \\ A_2 \end{pmatrix} B = \begin{pmatrix} A_1 B \\ A_2 B \end{pmatrix}$$

Доказательство. Очевидно. Действительно, пусть A_1 и A_2 имеют размеры $m_1 \times n$ и $m_2 \times n$ соответственно; рассмотрим (i, j)-тую клетку результата при $i \leq m_1$. Тогда в левой части мы возьмём i-тую строку подматрицы A_1 и умножим на j-й столбец B. В правой же части i-тая строка — это i-тая строка подматрицы $A_1 B$. Её j-тый элемент по определению получается при умножении i-той строки A_1 и j-того столбца B. Аналогично при $i > m_1$. $\square$

Блочное умножение

Следствие 1

$$\begin{pmatrix} A_1 \\ \vdots \\ A_r \end{pmatrix} B = \begin{pmatrix} A_1 B \\ \vdots \\ A_r B \end{pmatrix}$$

Действительно, применим лемму 1 к блокам $\begin{pmatrix} A_1 \\ \vdots \\ A_{r-1} \end{pmatrix}$ и A_r, потом к блокам $\begin{pmatrix} A_1 \\ \vdots \\ A_{r-2} \end{pmatrix}$ и A_{r-1} и так далее.

Блочное умножение

Лемма 2.

$$A \begin{pmatrix} B_1 & B_2 \end{pmatrix} = \begin{pmatrix} AB_1 & AB_2 \end{pmatrix}$$

(Доказательство полностью аналогично лемме 1)

Следствие 2

$$A \begin{pmatrix} B_1 & B_2 & \dots & B_t \end{pmatrix} = \begin{pmatrix} AB_1 & AB_2 & \dots & AB_t \end{pmatrix}$$

Блочное умножение

Лемма 3.

$$\begin{pmatrix} A_1 & A_2 \end{pmatrix} \begin{pmatrix} B_1 \\ B_2 \end{pmatrix} = A_1B_1 + A_2B_2,$$

где A_1 имеет размер $m \times n_1$, A_2 — $m \times n_2$, B_1 — $n_1 \times k$, B_2 — $n_2 \times k$.

Доказательство. Рассмотрим (i, j)-тую клеточку результата. Она равна

$$a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in_1}b_{n_1j} + a_{i(n_1+1)}b_{(n_1+1)j} + \cdots + a_{i(n_1+n_2)}b_{(n_1+n_2)j} =$$

$$(a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in_1}b_{n_1j}) + \left(a_{i(n_1+1)}b_{(n_1+1)j} + \cdots + a_{i(n_1+n_2)}b_{(n_1+n_2)j}\right).$$

Первая сумма — это в точности (i, j)-тая клеточка A_1B_1, вторая сумма — та же клеточка A_2B_2. □

Блочное умножение

Теперь можно аналогично предыдущему написать Следствие 3. Из предыдущих лемм и следствий вместе взятых очевидно следует наша теорема.

Применения блочного умножения

Наивный алгоритм умножения матриц

```
for i in range(0, m):
    for j in range(0,k):
        for s in range(0,n):
            c[i,j] += a[i,s]*b[s,j]
```

работает за $O(mnk)$ операций, если для простоты ограничиться квадратными матрицами, то есть положить $m = n = k$, то это $O(n^3)$ операций.

К сожалению, если матрицы довольно большие, то это много; хотелось бы ускорить процесс. Не удастся ли как-то помочь при помощи разбиения на блоки?

Довольно легко увидеть, что при разбиении на блоки каждое конкретное умножение всё равно будет произведено, просто в разные моменты времени...

Применения блочного умножения

Но это даёт неожиданное преимущество на более низком уровне. В строке `c[i,j] += a[i,s]*b[s,j]` не только производится одно умножение и одно сложение, но ещё считываются данные из оперативной памяти в кэш процессора и обратно. Если считать, что количество памяти не ограничено, то проблемы тут нет — объём данных равен $3n^2$ (если принять одну клетку матрицы за условную единицу измерения памяти), что при больших n гораздо меньше, чем $O(n^3)$, так что почти всё время будет уходить на арифметические операции (flops), а не операции с памятью (memops).

Но если количество памяти меньше, чем $3n^2$, то матрицы A, B, C не будут целиком помещаться в кэш. Оказывается, что тогда как минимум одна из них будет то считываться, то стираться и заново считываться, и количество операций с памятью возрастёт до $O(n^3)$. Оказывается, блочное умножение позволяет преодолеть эту проблему (мы не будем углубляться в детали).

Метод Штрассена

Другое красивое применение — метод Штрассена. Основная идея состоит в том, что если для какого-то конкретного размера матриц мы научимся умножать их быстрее, чем по определению, то дальше можно рекурсивно применять этот метод, разбивая матрицу на блоки. Фолькер Штрассен в 1969 году заметил (на основе уже существовавшего метода Карацубы для умножения чисел), что матрицы 2×2 можно перемножить за 7 умножений вместо 8-ми. Конкретных способов существует довольно много, вот исходный:

$$D = (A_{11} + A_{22})(B_{11} + B_{22});$$

$$D_1 = (A_{12} - A_{22})(B_{21} + B_{22}); D_2 = (A_{21} - A_{11})(B_{11} + B_{12});$$

$$H_1 = (A_{11} + A_{12})B_{22}; H_2 = (A_{21} + A_{22})B_{11};$$

$$V_1 = A_{22}(B_{21} - B_{11}); V_2 = A_{11}(B_{12} - B_{22});$$

Метод Штрассена

$$D = (A_{11} + A_{22})(B_{11} + B_{22});$$

$$D_1 = (A_{12} - A_{22})(B_{21} + B_{22}); D_2 = (A_{21} - A_{11})(B_{11} + B_{12});$$

$$H_1 = (A_{11} + A_{12})B_{22}; H_2 = (A_{21} + A_{22})B_{11};$$

$$V_1 = A_{22}(B_{21} - B_{11}); V_2 = A_{11}(B_{12} - B_{22});$$

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} D + D_1 + V_1 - H_1 & V_2 + H_1 \\ V_1 + H_2 & D + D_2 + V_2 - H_2 \end{pmatrix}.$$

Обратите внимание, что хотя умножений становится 7 вместо 8, сложений становится 18 вместо 4. Есть модификации, немного уменьшающие число сложений.

Итоговая сложность алгоритма составляет $O(n^{\log_2 7})$, то есть $O(n^{2,81})$.